- **Fill-in-the-middle:** The FIM approach (Bavarian et al., 2022) moves a span of text from the middle to the end, and then trains the transformer with next token prediction. This can be seen as a generalization of the data augmentation approach, where subsequences of intermediate nodes are used in lieu of a single intermediate node.
- **Multi-token Prediction:** The multi-token approach refers to a Transformer trained to predict the entire path $\prod_{i=1}^{T} p(n_i \mid \mathcal{E}, n_1, n_T)$ in one forward pass without providing access to tokens from a partial path (hence the name "Teacherless" in Bachmann & Nagarajan (2024)).

Our code and baselines adopt the GPT2 Transformer architecture, using standard hyperparameter settings for number of layers, embedding dimension, etc. All models are trained on the same dataset with the same training budget. See Section C for additional training information and model setup.

3.5 Results

As seen in Figure 2, the Belief State Transformer successfully learns to solve all the graphs. Since the Belief State Transformer uses an empty suffix, the parameter counts at inference time are very similar with minor variations driven by variations in output heads.

The baselines have varying degrees of success. The forward only baseline achieves at most a $1/d$ success rate, as it learns to output valid paths at random during inference time. Similarly, the data augmentation and FIM baselines perform poorly. We suspect that despite access to additional information like intermediate sub-goals or paths, the gradients which the model is exposed to are still inadequate to encourage the development of appropriate representations due to the easy availability of shortcut solutions.

The multi-token baseline, while successful in some smaller graphs like $G(2, 5)$, fails to solve more complicated graphs with more arms ($G(5, 5)$) or with longer path lengths ($G(2, 20)$), reproducing prior results of Bachmann & Nagarajan (2024).

3.6 Belief State Transformer Variants

Next, we ablate the Belief State Transformer to characterize its performance. The **Belief w/o Prev** ablation removes the previous token decoder and its objective from the training. The **Belief w/o Backward** ablation removes the backward encoder B from the training and inference process.

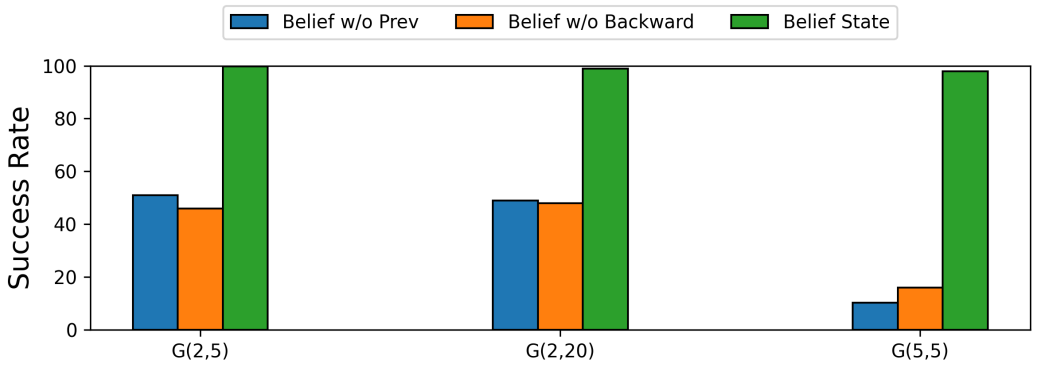


Figure 4: Ablations of the Belief State Transformer on the star graph. Both the belief state objective and the backward encoder are crucial.

The results in Figure 4 show that both the belief state objective and backward encoder are important components of the Belief State Transformer, and removing either drops performance back down to randomly guessing amongst valid paths. Prediction of the Prev token forces the Transformer to learn to represent long term dependencies which ends up being useful for goal-conditioned navigation.

4 BELIEF STATE TRANSFORMER ANALYSIS

In this section, we show that the Belief State Transformer discovers a compact belief state, and that the forward-only and multi-token approaches do not[1]. First, we formally define a belief state.

Definition 1 (Belief State)**.** *For any probability distribution over a set of sequences $P(x_{1:T})$, for any partial sequence $s = x_{1:t}$, a vector v_s is a belief state for s if there exists a randomized function $g(v_s)$ which can sample from the conditional distribution $\mathbb{P}(x_{t+1:T}|x_{1:t})$.*

By definition, a belief state captures all available information relevant for predicting the future tokens. Once the belief state is learned, there is no additional useful information to be gained—everything necessary for future predictions is already encoded within it.

4.1 BELIEF STATE DISCOVERY

We first show that successfully optimizing a Belief State Transformer results in a compact belief state.

Theorem 2. *Let $D = P(x_{1:T})$ represent any given probability distribution over a set of sequences. Consider an ideal Belief State Transformer that satisfies the following conditions for all prefixes $x_{1:t}$ and suffixes $x_{t+k+1:T}$:*

$$\mathsf{T}_n(x_{t+1} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k+1:T})) = \mathbb{P}(x_{t+1} \mid x_{1:t}, x_{t+k+1:T}), \tag{3}$$

$$\mathsf{T}_p(x_{t+k} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k+1:T})) = \mathbb{P}(x_{t+k} \mid x_{1:t}, x_{t+k+1:T}). \tag{4}$$

Then, for any partial sequence $x_{1:t}$ supported by D, the forward encoding of the ideal Belief State Transformer, $\mathsf{F}(x_{1:t})$, is a belief state for $x_{1:t}$.

Proof. Let $s_t = x_{1:t}$ denote the prefix. To prove that $f_t \coloneqq \mathsf{F}(s_t)$ is a belief state for s_t, we must show that, given f_t, one can sample from the conditional distribution $\mathbb{P}(x_{t+1:T} \mid s_t)$.

The key observation is that the conditional distribution can be decomposed as follows:

$$\mathbb{P}(x_{t+1:T} \mid s_t) = \mathbb{P}(x_T \mid s_t)\mathbb{P}(x_{T-1} \mid s_t, x_T) \cdots \mathbb{P}(x_{t+1} \mid s_t, x_{t+2:T}).$$

For an ideal Belief State Transformer, this decomposition can be rewritten as:

$$= \mathsf{T}_p\big(x_T \mid f_t, \mathsf{B}(\emptyset)\big) \cdot \mathsf{T}_p\big(x_{T-1} \mid f_t, \mathsf{B}(x_T)\big) \cdots \mathsf{T}_p\big(x_{t+1} \mid f_t, \mathsf{B}(x_{t+2:T})\big),$$

Thus, using the forward encoding f_t, one can generate the remaining sequence $x_{t+1:T}$ by sampling in reverse order—first sampling x_T, then x_{T-1} conditioned on x_T, and so on, until x_{t+1}. Each step involves using the Prev decoder and updating the backward encoder with the newly generated token.

Since the forward encoding f_t enables sampling from the conditional distribution $\mathbb{P}(x_{t+1:T} \mid s_t)$ in this way, it follows that f_t is a belief state for s_t. □

4.2 NEXT-TOKEN OR MULTI-TOKEN PREDICTION DOES NOT GUARANTEE BELIEF STATES

Theorem 2 establishes that an ideal Belief State Transformer learns correct belief states. In contrast, the following two theorems demonstrate a fundamental limitation of standard next-token predictors and their variants when viewed from the belief state perspective.

Theorem 3. *Consider a standard next-token predictor with a forward encoder F and output head T such that*

$$\mathsf{T}(\mathsf{F}(x_{1:t})) = \mathbb{P}(x_{t+1} \mid x_{1:t}). \tag{5}$$

There exists a distribution $P(x_{1:t})$ over sequences and a next-token predictor of the form Equation (5) such that the input to the output head is not a belief state.

The proof is provided in Section B.2. Next, we analyze the case of multi-token training (see Section 3.4), where the model is asked to predict multiple future tokens at once.

[1]The fact that the forward-only approach does not produce a compact belief state is well known. We formalize it here to contrast with the Belief State Transformer.

Theorem 4. *Consider the multi-token setting where, for predicting H tokens into the future, the model is of the form*

$$\mathsf{T}_j(\mathsf{F}(x_{1:t})) = \mathbb{P}(x_{t+j} \mid x_{1:t}) \quad \textit{for } j = 1, 2, \ldots, H. \tag{6}$$

There exists a distribution $D = P(x_{1:t})$ over sequences and a multi-token model satisfying Equation (6) *such that the input to the output head is not a belief state.*

The proof is provided in Section B.3. In summary, the analysis in this section highlights the inherent limitations of next-token predictors: the input to the output head fails to capture the belief state.

5 Experimenting with the Belief State Transformer

Given a model jointly trained on prefixes and suffixes, the representation is useful in new ways which are not available to simple forward Transformers. Here we detail two different forms of search based on forward and backward probabilities (respectively) as well as belief state embedding extractions.

Setup. We use TinyStories (Eldan & Li, 2023), a dataset consisting of synthetic short stories. TinyStories aims to represent key challenges in text generation while keeping training tractable for small to medium scale models. We tokenize the dataset into a vocabulary space of size 1000, and discard stories greater than 256 tokens long resulting in a dataset consisting of 2.7 million stories.

During evaluation, the models generate text using prefix-suffix snippets from an evaluation set of 100 unseen stories. Given stories from two competing models, GPT4 is then asked to output an analysis of each story examining multiple factors (e.g. grammar, flow, cohesiveness, creativity) before outputting a final recommendation. We follow best practices in evaluating with multiple trials and shuffling choice order. We report the winrate and confidence interval (CI) for each model. See Section D.2 for more details and examples of the GPT4 judge outputs and scoring.

5.1 Goal-conditioned Text Generation

In the goal-conditioned setting, the user provides the model with a prefix and suffix, and the model infills the text in between. See below for an example of the prefix and suffix. We describe a goal-conditioned planning procedure with the Belief State Transformer for text generation in Algorithm 1.

Method. The algorithm performs n roll-outs. In each roll-out (starting at line 3), a candidate trajectory is generated that differs from the greedy trajectory but maintains high probability. The key features of the process are as follows:

1. *Greedy generation*: Starting with a sequence s popped from the priority queue Q, a candidate trajectory is extended up to a maximum length k using argmax greedy selection (line 6). The completed trajectory is appended to the set of candidates C (line 11).

2. *Priority queue update for future generation*: Simultaneously, the priority queue Q is updated to track alternative candidate sequences. This is done by appending non-greedy tokens to the queue, with their priority set by the *relative suboptimality* of each token. Specifically, lines 8 and 9 add alternative tokens with priority equal to the current sequence priority multiplied by the ratio of the alternative token's probability to that of the greedy token. Since this ratio is ≤ 1, priorities decreases with suboptimality. Also, this ensures that partial sequences of different lengths remain comparable, as suboptimality is independent of sequence length. This encourages branching at ambiguous points.

Algorithm 1 Goal-conditioned Planning

Require: prompt $x_{1:t}$, goal $x_{t+k:T}$, horizon k, rollouts n

1: Priority Queue $Q \leftarrow (1, x_{1:t})$
2: Candidates $C \leftarrow \emptyset$
Roll-outs start here
3: **for** $j = 1, \ldots, n$ **do**
4: (priority r, sequence s) $\leftarrow$ pop(Q)
5: **while** $|s| < t + k - 1$ **do**
6: *Greedy generation*
$x_{\max} \leftarrow \arg\max_x \mathsf{T}_n(x \mid \mathsf{F}(s), \mathsf{B}(x_{t+k:T}))$
7: *Priority queue update for future generation*
value $\mathsf{T}_{\max} = \mathsf{T}_n(x_{\max} \mid \mathsf{F}(s), \mathsf{B}(x_{t+k:T}))$
8: **for** $x \neq x_{\max}$ **do**
9: $Q \leftarrow (r \cdot \frac{\mathsf{T}_n(x \mid \mathsf{F}(s), \mathsf{B}(x_{t+k:T}))}{\mathsf{T}_{\max}}, s + x)$
10: $s \leftarrow s + x_{\max}$
11: *Appending C with greedy generation*
$C \leftarrow C \cup \{s + x_{t+k:T}\}$
12: *Scoring*
Output: $\arg\max_{x_{1:T} \in C}$ of Equation (7)

3. *Scoring*: Once candidate trajectories are generated, they are scored by evaluating the consistency of generated tokens $x_{t+1:t+k-1}$ with the goal $x_{t+k:T}$ using the next-head probability:

$$\prod_{i=t+k}^{T} \mathsf{T}_n(x_i \mid \mathsf{F}(x_{1:i-1}), \mathsf{B}(x_{i+1:T})) \tag{7}$$

We then return the highest-scoring trajectory based on Equation (7).

Baseline. We select the Fill-in-the-middle (FIM) (Bavarian et al., 2022) approach as a natural goal-conditioned baseline. FIM trains a single forward-only transformer where the input is the concatenated suffix and prefix, and the objective is next token prediction. Because the forward-only approach is radically less efficient, we sample suffixes uniformly at random during training. During inference, beam search (Graves, 2012) is performed to search for high-probability sequences.

For fair comparisons, we train the FIM and Belief State Transformer with a similar amount of resources in terms of wall-clock and GPU usage, and set the architecture configuration so that the number of parameters between models is similar (80M for Belief, 85M for FIM). During inference time, we configure the search methods so that the computational budget is similar, i.e. setting the number of beams and depth to the values we use in Algorithm 1. See Section D.1 for more details.

Results. As seen in Figure 5, GPT4 prefers the Belief State transformer 64% of the time with a binomial confidence interval of $[54\%, 74\%]$. Qualitatively, the stories of Belief State Transformer are superior to FIM in several ways. The Belief State outputs more frequently display clear narrative structure, with a beginning, middle, and end, while the FIM model tends to generate shorter stories with simpler structure and repetitions. The FIM model also frequently failed to connect its completions to the suffix, resulting in abrupt and grammatically incorrect transitions. See below for an example.

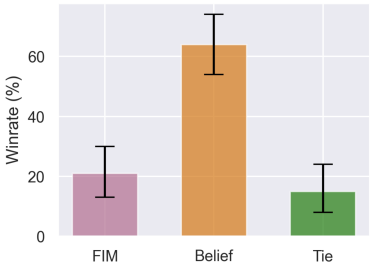


Figure 5: Storytelling winrates.

Prefix: *Once upon a time, there was a king. He was a big and strong king who ruled over his kingdom. One day, he wanted to take a nice and long bath, so he filled the bathtub with warm water*

Belief: . He got in and started to wash. The king was very busy, but he was so busy that he didn't notice the water was getting too hot. Suddenly, he felt a sharp pain in his foot. He looked down and saw it burning. He was very scared and he quickly put his foot in the water. The king was very worried. He

FIM: and lots of bubbles. He stepped into the bathtub and started to play with the bubbles. Suddenly, the king noticed that one of the bubbles was getting bigger and bigger. He asked the king, "What's happening?" The king replied, "The bubble is getting bigger and bigger. It's getting

Suffix: *quickly grabbed a cloth and began to clean it up. The king got so hot from cleaning up the mess that he decided to take another soak in the bathtub. He put a lot of bubbles in the water to make it nice and bubbly. He relaxed again and felt all the worries wash away.*

In this example, the Belief State Transformer's story has a clear narrative structure with setup, conflict and resolution. It is also able to correctly connect the prefix, generated text, and suffix. On the other hand, the FIM model has repetitions ("bigger and bigger"), confusing dialogue where a previously unmentioned person asks the king a question, and an abrupt and grammatically incorrect transition to the suffix ("It's getting quickly grabbed"). See Sections D.2 and D.3 for more examples as well as the grade reports and evaluation outputs of the GPT4 judge.

5.2 Unconditional Text Generation

Method. Next, we investigate the unconditional setting, or when the goal is unknown. We reuse the logic in Algorithm 1 with two following changes: setting the goal to the empty input $z'_{\emptyset} := \{\emptyset\}$, and scoring the sequences with the previous head T_p over a fixed amount of suffix tokens k. We choose to use the previous head rather than the next head as a semi-independent evaluator of next-generated tokens to reduce the bias associated with self-evaluation. By scoring over the last k tokens, this selects for trajectories whose endings are more likely.

$$\prod_{i=T-k}^{T} \mathsf{T}_p(x_i \mid \mathsf{F}(x_{1:i-1}), \mathsf{B}(x_{i+1:T})) \tag{8}$$